Transilvania
University
of Brasov

Transilvania
University
of Brasov
FACULTY OF ELECTRICAL ENGINEERING
AND COMPUTER SCIENCE

Department of Electronics and Computers
Study programme: Applied Electronics

VERIFICATION PROJECT
ALU with internal memory

Table of Contents

● DIGITAL DESIGN OF THE CIRCUIT

●.1 BLOCK DIAGRAM

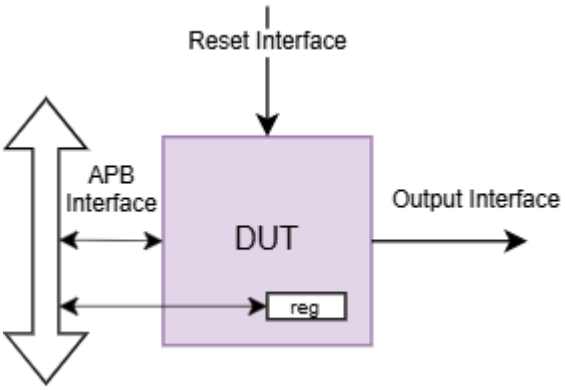


Figure 1. Interfaces of the ALU.

According to the schematic in Figure 1., the data transmission interface is bidirectional, because the DUT transmits the data through the APB interface, and receives data through the same interface. The STATE, which represents the state of the DUT, is part of the reset interface and can take only two values, RUN and FREEZE. The OUTPUT interface represents the Result of the Operation.

●.2 GENERAL DESCRIPTION OF DUT FUNCTIONALITY

The DUT supports several operations as its functionality, either mathematical operations orcompare operations.

- ❖ Write and read functions:
  - ➢ Thanks to the APB interface and the internal memory, the DUT can store values in registers (up to 16). On read, the DUT fetches the register at the address sent over APB and presents the register value on the Output.

- ❖ Operations:
  - ➢ Addition: Depending on the opcode received through the APB interface, the addition has 3 forms (Operand1 + Operand2, Operand1 + Constant, Operand2 + Constant).
  - ➢ Shift: The shift is always applied to Operand1 based on the shift value taken from pwdata.
  - ➢ OR and NOR: Bitwise OR/NOR logic gate applied between the operands.
  - ➢ AND and NAND: Bitwise AND/NAND logic gate applied between the operands.
  - ➢
  - ➢ Compare: Depending on the opcode received through the APB interface, there are 3 compare variants applied between the operands (Operand1 == Operand2, Operand1 < Operand2, Operand1 > Operand2).
- ❖ Error detection:
  - ➢ Because of possible future DUT extensions, the address is 6 bits wide, but the internal registers only cover decimal values up to 15. Therefore, on a read or write to a higher address, the DUT asserts the PSLVERR signal on the APB interface.

●.3 DUT INTERFACES

The table is filled with the DUT interfaces. Signals are grouped by the interface they belong to. A separate signal table is produced for each interface. Signal direction is considered from the DUT's point of view (a signal carrying data into the DUT is an input).

Inputs:

- clk: clock signal.
- reset_n: reset signal, active low.
- psel, penable: APB control signals.
- paddr: address on the APB bus (used to read back results).
- pwrite: write signal.
- pwdata: input data written through APB.
- state: external signal that enables operation execution only while the ALU is in the RUN.

Outputs:

- prdata: data returned after a read.
- pready: APB operation valid signal.
- pslverr: error signal for an invalid address.

- result: current computed result output (visible only while an operation is active).

●.1 LIST OF DUT FUNCTIONALITIES

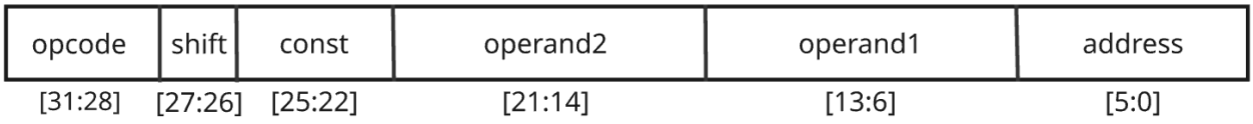
The module ALU_control represents the control unit of an arithmetic-logic unit (ALU) with internal memory, capable of performing various arithmetic and logic operations, all accessed through the APB (Advanced Peripheral Bus) bus. This module controls the process of loading the operand(s) and the constant, executes the specified operations and stores the result in internal addressable registers.List of scenarios designed for the DUT verification

The module is organized around a **finite state machine (FSM)** with distinct states for each operation and intermediate control step:

Defined states:

- INIT: reset / init state.
- SETUP: extracts the operand(s), the constant and the operation type from received_data.
- ADD1, ADD2, ADD3: arithmetic operations (between op1 / op2 / const).
- SHIFT_OP1: left shift applied to op1.
- AND, OR, NAND, NOR: logical operations between op1 and op2.
- COMP: compare between op1 and op2.
- ENDOP: operation completion and reset of signal init.

When pwrite=1, the DUT writes into received_data a 32-bit word, interpreted as follows:



- [31:28] – operation code
- [27:26] – shift amount bits
- [25:22] – constant (4 bits)
- [21:14] – operand2 (8 bits)
- [13:6] – operand1 (8 bits)
- [5:0] – address where the result is stored

Operation code (`[31:28]`)	Operation description	Formula
`0001` `(ADD1)`	Arithmetic: `op1 + op2`	`result = op1 + op2`
`0010` `(ADD2)`	Arithmetic: `op1 + const`	`result = op1 + const`
`0011` `(ADD3)`	Arithmetic: `op2 + const`	`result = op2 + const`
`0100` `(SHIFT_OP1)`	Left shift of `op1`	`result = op1 << shift_amount`
`0101` `(AND)`	Logic: `op1 & op2`	`result = op1 & op2`
`0110` `(OR)`	Logic: `op1 | op2`	`result = op1 | op2`
`0111` `(NAND)`	Logic: `~(op1 & op2)`	`result = ~(op1 & op2)`
`1000` `(NOR)`	Logic: `~(op1 | op2)`	`result = ~(op1 | op2)`
`1001` `(COMP)`	Comparison between `op1` and `op2`	`result = 1 (>) / 2 (<) / 256 (==)`

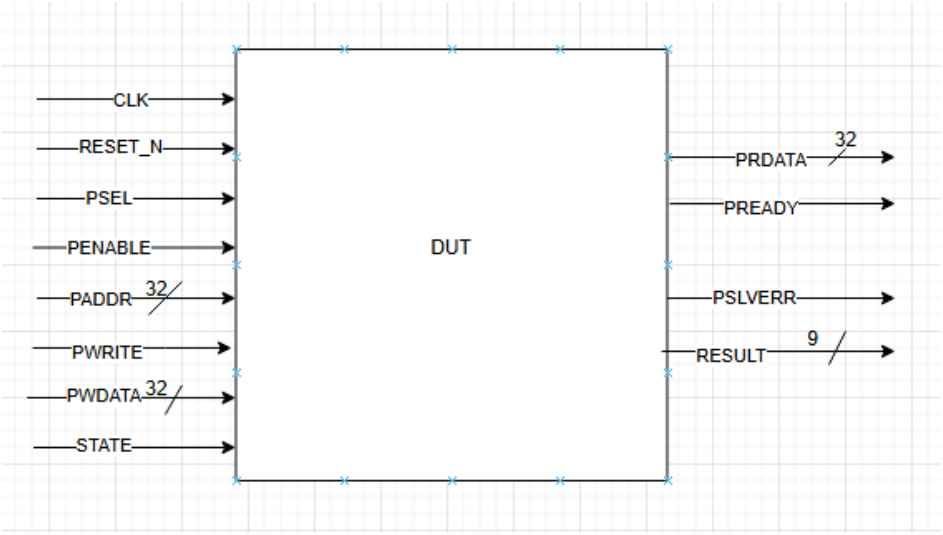
Operation

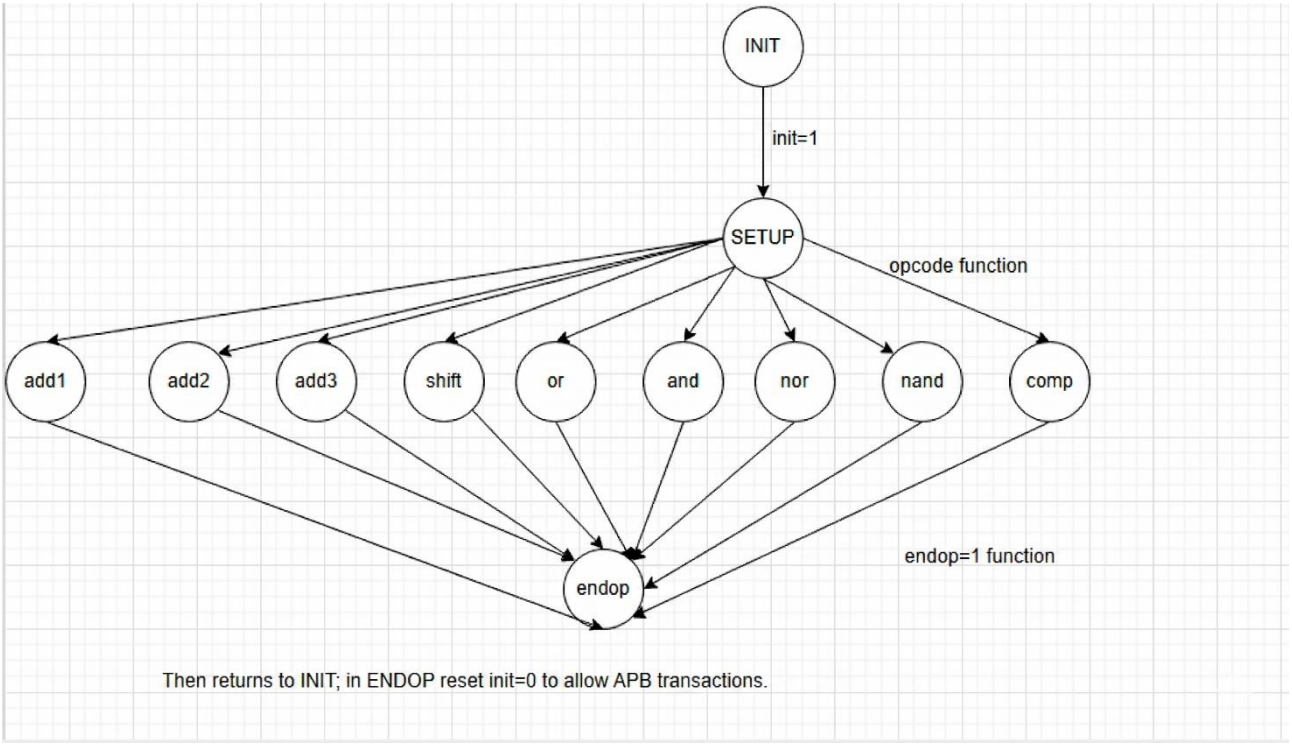
1. The command data is written through APB (pwdata), when psel, penable, pwrite are active and state=1.
2. In the SETUP state the instruction is decoded and an operation state is entered.
3. The result is computed and stored in register result.
4. The value of result is saved into one of the 16 registers (ResultReg_0 up to ResultReg_15) depending on the address specified in [5:0].
5. Later, the stored value can be read from that address using paddr.

Protection elements

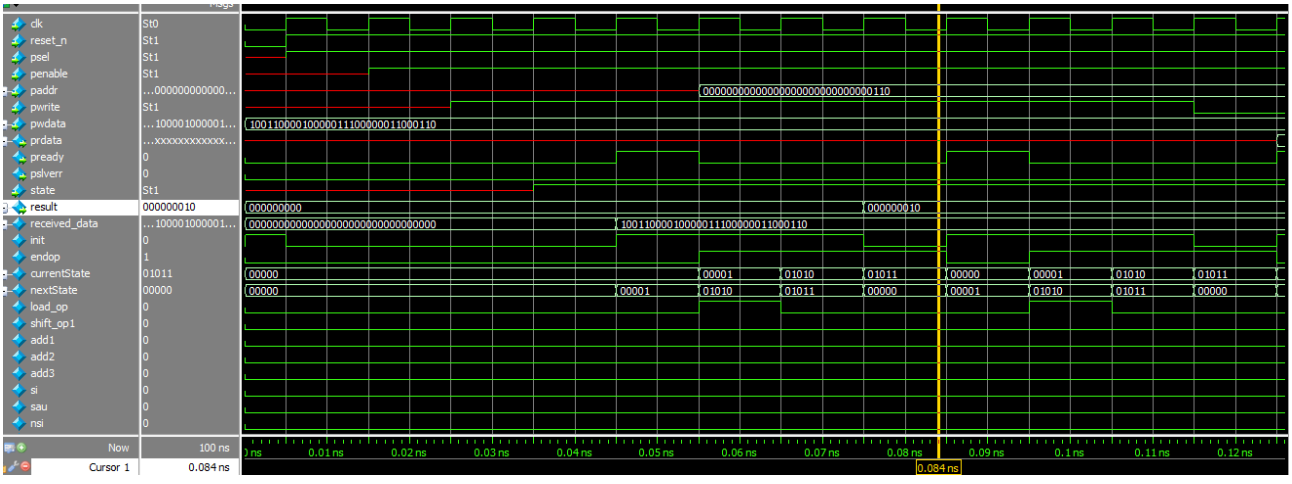
- If an invalid address (> 15) is used when storing the result, signal pslverr is asserted.
- The reset (reset_n=0) brings all internal variables back to their initial state.

●.2 PRINCIPAL SCHEMATIC





●.3 WAVEFORMS



●.4 REGISTER TABLE

Table 4. Registers accessible by the DUT

Register name	Access address	Register layout	Reset value	Register field description
received_data	d16	32b	d0	Configurable register, see the per-field description above; both write and read are supported
ResultReg_0	d0	9b	d0	Result-storage register, read-only
ResultReg_1	d1.	9b	d0	Result-storage register, read-only
ResultReg_2	d2	9b	d0	Result-storage register, read-only
ResultReg_3	d3	9b	d0	Result-storage register, read-only
ResultReg_4	d4	9b	d0	Result-storage register, read-only
ResultReg_5	d5	9b	d0	Result-storage register, read-only
ResultReg_6	d6	9b	d0	Result-storage register, read-only
ResultReg_7	d7	9b	d0	Result-storage register, read-only
ResultReg_8	d8	9b	d0	Result-storage register, read-only
ResultReg_9	d9	9b	d0	Result-storage register, read-only
ResultRe	d10	9b	d0	Result-storage register,

g_10				read-only
ResultReg_11	d11	9b	d0	Result-storage register, read-only
ResultReg_12	d12	9b	d0	Result-storage register, read-only
ResultReg_13	d13	9b	d0	Result-storage register, read-only
ResultReg_14	d14	9b	d0	Result-storage register, read-only
ResultReg_15	d15	9b	d0	Result-storage register, read-only

● VERIFICATION OF THE DIGITAL CIRCUIT DESIGN

Verification environment architecture

List of items to verify

Functional coverage elements

Verification scenarios and tests

Results obtained

●.1 VERIFICATION ENVIRONMENT ARCHITECTURE

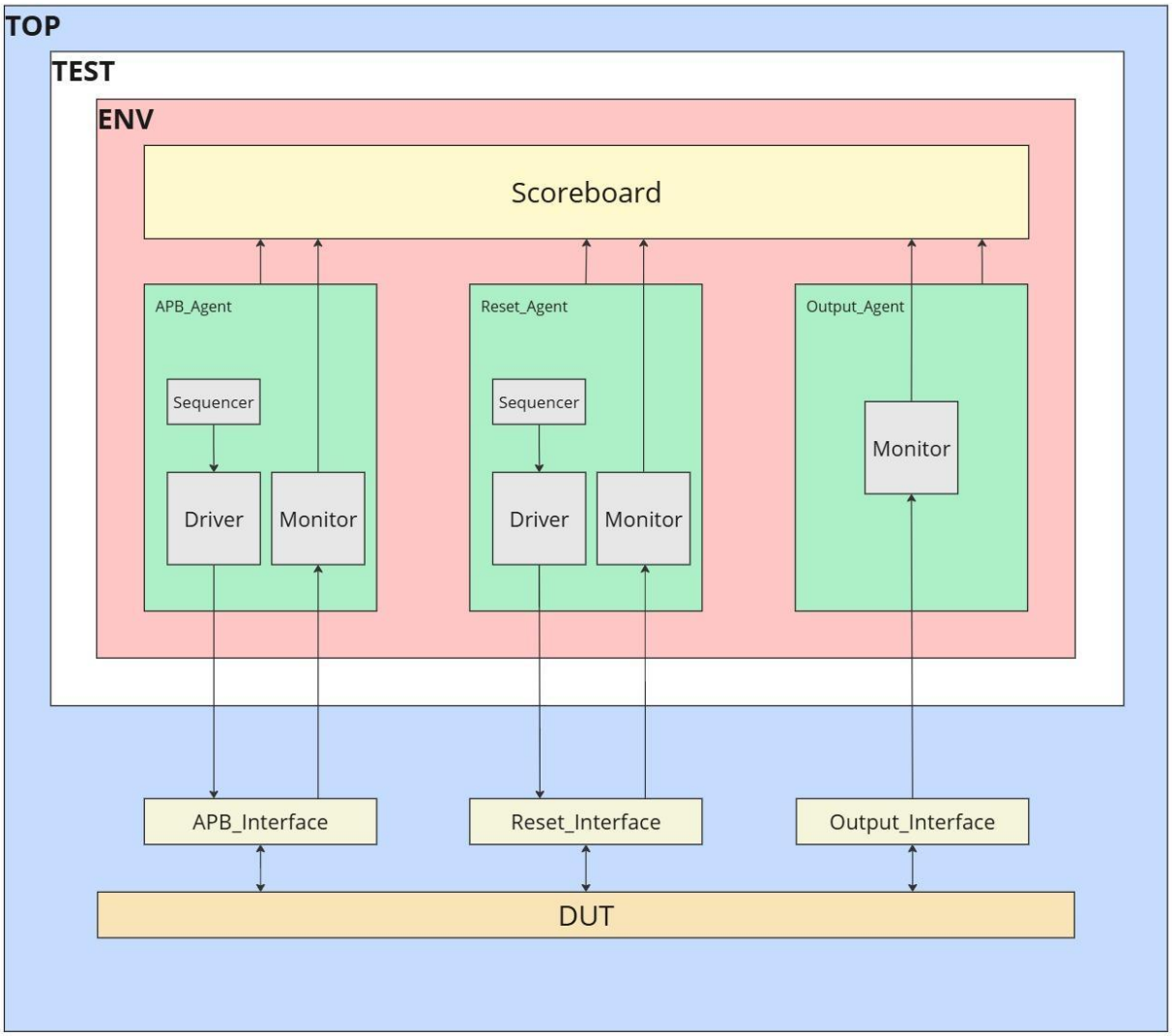


Figure 3. Example overview of the verification environment

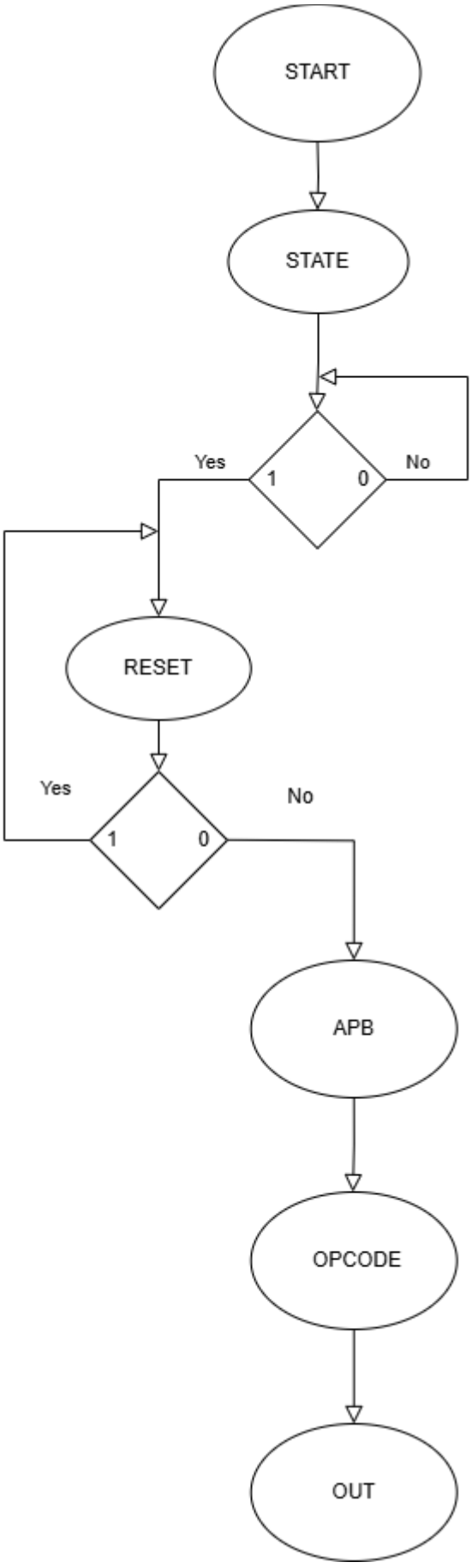


Figure 4. Example of a block diagram

●.1.1 Reset Driver

1. General Description

This driver implements the logic that generates and applies the reset signal to the modules of a UVM testbench. reset_driver takes reset_item objects from sequences and turns them into active stimuli, driving reset_n towards the design under test through a virtual interface. The driver makes it possible to test the system's behavior in reset conditions; it is responsible for applying the reset signal correctly and in sync with the scenarios defined in the sequences.

2. Signals

reset_n = 0 – the system is held in reset.

reset_n = 1 – the system leaves reset and operates normally.

The driver uses this signal to simulate behaviors such as:

- Synchronous/asynchronous reset.
- Testing behavior on exit / sudden release from reset.
- Successive resets during execution.

3. Driver Functionalities

The reset_driver implements the essential functionality of generating and applying the reset signal inside a UVM testbench. This functionality is essential for robust system testing: it simulates different initialization scenarios and post-reset behaviors.

Reset application

Applies the reset signal (reset_n) to the design under test based on the value received from the attached sequence.

- Takes reset_item objects from the sequence.
- Writes the reset_n value to the virtual interface connected to the DUT.
- Simulates initialization behavior or multiple resets during execution.

4. Workflow

Initialization Phase

- In build_phase, the driver retrieves the reset_interface virtual interface through the uvm_config_db mechanism.
- If the interface is not set correctly, the driver stops the test with an error message (uvm_fatal).

Reset Execution Phase

- In run_phase, the driver runs continuously in a (forever) loop, waiting for reset_item items.
- For each item:
    - Retrieves the reset_n value from the item.
    - Applies this value to the reset_n signal on the virtual interface.
    - Marks the item as processed (item_done).

- Prints debugging information (sprint() and uvm_info).

Completion

- After applying the reset, the driver stays active and waits for new items so that it can simulate further reset conditions if needed.

5. Usage Examples

The driver can simulate the following types of scenarios:

Initial reset

At the start of a simulation, the sequence generates an item with reset_n = 0, then reset_n = 1, thus simulating a full system reset.

Reset during execution

During the test, additional items with reset_n = 0 can be injected to simulate unexpected resets and test the system's resilience.

6. Driver Block Diagram

The driver follows a simple, reset-specific logic flow:

- Wait for item: the driver waits for a new reset_item object from the sequence.
- Apply reset: the reset_n field value is driven towards the DUT.
- Mark completion: the driver notifies the sequence that the item was processed.
- Continuous loop: the process repeats for every new item received.

●.1.1 APB Driver

1. General Description

This driver implements the APB (Advanced Peripheral Bus) protocol in order to let the Verilog testbench interact with peripheral modules that use this bus. The driver performs read and write transactions to and from APB slave modules, ensuring correct signal synchronization and protocol compliance.

2. Driver Signals

- **psel**: APB slave select signal. Asserting this signal indicates that the driver will start a transaction with the selected slave.
- **penable**: Signal indicating that the transaction has been enabled. It marks the moment when the actual transaction takes place on the APB bus.
- **pwrite**: Transaction direction signal. Value 1 indicates a write transaction, while 0 a read transaction.
- **paddr**: Memory address being accessed during the APB transaction.
- **pwdata**: Data to be written on the APB bus (for a write transaction).
- **prdata**: Data read from the APB slave (for a read transaction).
- **pready**: Signal indicating that the APB slave is ready to complete the transaction.

- **pslverr**: Signal indicating an error within the APB transaction.

3. Driver Functionalities

The driver implements the fundamental read and write functionalities according to the APB protocol:

APB Write

- Allows data to be written at a given address on the APB bus.
- Sets the appropriate signals to start a write transaction, wait for the completion signal pready and deassert the transaction at the end.

APB Read

- Allows data to be read from a given address on the APB bus.
- Sets the signals needed to start a read transaction and waits for the APB slave to become ready to send the data.

4. Workflow

Address Setup Phase

- The driver sets signal **psel** to 1 to signal that the transaction is active.
- **penable** is set to 0 to mark the start of the transaction.
- **paddr** is set to the read or write address.
- For a write transaction, **pwrite** is set to 1 and **pwdata** is set to the data to be written.

Transaction Execution Phase

- Signal **penable** is set to 1 to start the data transfer.
- The driver waits for signal **pready** to confirm that the slave has processed the transaction and is ready to finish.
- For a read transaction, the driver waits for the data to become available on signal **prdata**.

Transaction Completion

- If the transaction completes successfully, the driver sets **psel** and **penable** to 0 to end the transaction.
- If an error is detected during the transaction (via signal **pslverr**), it will be reported as an error.

5. Usage Examples

The driver supports two main types of transactions: write and read.

APB Write

A data set can be written at a specified address by calling the driver with the matching parameters: address and data to write.

APB Read

A data set can be read from a specified address, and the read data is returned via an output variable.

6. Driver Block Diagram

The driver follows a simple logic flow to start and handle APB transactions, following these steps:

- **Set psel to 1**: the select signals are activated to mark the start of the transaction.
- **Set penable to 0**: the transaction is configured before activation.
- **Set address and data** (for a write).
- **Set penable to 1**: the transaction becomes active.
- **Wait for pready**: the driver waits for confirmation that the transaction has completed.
- **Transaction completion**: at the end of the transaction, the signals are reset to return to the initial state.

●.1.1 Scoreboard / reference model

The scoreboard is responsible for verifying the functional correctness of the DUT. It receives transactions of type apb_item, output_item and reset_item through **Analysis Ports**, and compares the results produced by the DUT with those computed internally using its own logic.

```
class scoreboard extends uvm_scoreboard;
```

It is a class derived from uvm_scoreboard, registered with the macro:

```
`uvm_component_utils(scoreboard)
```

Three Analysis Imports are also declared, each one specialized for a different transaction type:

```
`uvm_analysis_imp_decl(_apb_port)
`uvm_analysis_imp_decl(_output_port)
`uvm_analysis_imp_decl(_reset_port)
```

- ap_imp_apb: receives transactions from the APB monitor

- ap_imp_output: receives the monitored result of the operations executed by the DUT
- ap_imp_reset: receives reset signals from the testbench

Internal variables

```
bit [5:0]   pwdata_address;
bit [7:0]   pwdata_operand1, pwdata_operand2;
bit [3:0]   pwdata_constant, pwdata_opcode;
bit [1:0]   pwdata_shift;
bit [5:0]   addres_read;
bit [8:0]   Resultmem[16];
bit [8:0]   ResultOutput;
bit         reset;
```

- They are used to decode the fields from pwdata and to simulate the ALU's behavior in memory.
- Resultmem is a simulated internal memory used to store the results of write operations.
- ResultOutput is the last computed result.

Key methods

write_apb_port(apb_item item)

This function handles both the write (pwrite == 1) and the read (pwrite == 0) on the APB interface:

- **For write:**
  - The fields from item.pwdata are decoded.
  - The operation code (opcode) is checked and the result is computed per the instruction (ADD, AND, SHIFT, etc.).
  - The result is stored in the virtual memory Resultmem.
- **For read:**
  - Compare prdata against the expected value from Resultmem[addres_read].
  - If the values do not match, an error is reported with uvm_error.

write_reset_port(reset_item item_reset)

- If signal reset_n is active (logic 0):
  - The entire virtual memory and all internal variables are reset.
  - The reset values are logged for traceability.

write_output_port(output_item item_output)

- The code is currently commented out.
- The intent is to compare item_output.result against ResultOutput, in order to detect discrepancies between what the DUT monitor sees and what the scoreboard computes.

Supported operation codes

OPCODE	Operation	Description
`0001`	`ADD1`	`op1 + op2`
`0010`	`ADD2`	`op1 + constant`
`0011`	`ADD3`	`op2 + constant`
`0100`	`SHIFT_OP1`	`op1 << shift`
`0101`	`AND`	`op1 & op2`
`0110`	`OR`	`op1 | op2`
`0111`	`NAND`	`~(op1 & op2)`
`1000`	`NOR`	`~(op1 | op2)`
`1001`	`COMP`	comparison between `op1` and `op2`

UVM Messages

- uvm_info – for logging at different detail levels (LOW, MEDIUM)
- uvm_error – to report errors such as:
  - Address out of range (max 15)
  - Mismatch between prdata and the value from memory
  - Invalid opcode (0000)

Remarks

- The scoreboard simulates the ALU's functionality using the same rules as the DUT, which makes it effective at catching functional errors.
- It is fully independent of the DUT and provides a source of truth ("golden model") inside the testbench.
- Verification based on reset is a significant plus for test stability.

●.1.1 APB Monitor

The APB monitor is a passive UVM component that observes transactions on the APB bus without influencing them. It:

- Connects to the virtual interface apb_interface through the UVM config DB.
- Monitors signals psel, penable and pready to detect valid APB transactions.
- Captures input/output data depending on signal pwrite (write or read).
- Measures the delay between transactions using signal psel.
- Sends the apb_item (which holds the transaction data and the delay) to analysis port mon_analysis_port_apb for coverage collection or scoreboard verification.

●.1.1 Monitor for the RESET interface

The RESET monitor is a passive UVM component that observes the reset signal in the design without influencing it. It:

- Connects to the reset_interface virtual interface through the UVM config DB, using the standard uvm_config_db mechanism to obtain the interface reference.
- Monitors the reset_n transitions to detect reset release events (posedge on reset_n).
- Creates reset_item objects to represent every detected reset event.
- Sends the reset_item objects through the mon_analysis_port_reset analysis port to other UVM components such as the coverage collector or the scoreboard for further analysis.
- Emits informative messages using uvm_info to indicate that the reset event was detected and forwarded correctly.

●.1.1 Transaction for the APB interface

An APB transaction represents a **read or write** operation applied to one of the ALU module's internal registers through the APB interface. Every transaction includes:

- **The** target register address (PADDR) – identifies the control area (e.g.: operand, result, state, opcode, etc.).
- **Operation type** (PWRITE) – indicates whether it is a write (1) or a read (0).
- **Data to write** (PWDATA) – used only in write transactions.
- **Data read** (PRDATA) – returned by the ALU in read transactions.
- **Control signals** (PSEL, PENABLE, PREADY) – coordinate the transaction flow according to the APB protocol.

The transactions are detected and analyzed by **the APB monitor**, which extracts this information and uses it for functional verification (scoreboard) and coverage.

●.1.2 Transaction for the RESET interface

The reset_item transaction object represents a reset event observed by the reset_monitor. Because the reset signal (reset_n) is, as a rule, a binary control signal (asserted/deasserted), the transaction does not hold complex data fields and mainly serves as a notification of a reset event.

●.1.3 Agent for the RESET interface

The RESET agent is the UVM component responsible for managing the reset interface inside the testbench. It encapsulates the three main components associated with a standard UVM interface:

- reset_driver – sends transactions to the reset_interface (active mode only).
- reset_monitor – observes the reset_n signal and extracts reset events for verification.
- reset_sequencer – receives transactions generated by sequences (uvm_sequence) and forwards them to the driver.

Operating modes

- Active (UVM_ACTIVE) – the agent instantiates all three components (driver, monitor, sequencer) and connects the sequencer to the driver.

UVM phases used

- build_phase: instantiates the agent components depending on the active/passive mode. If the agent is active, it creates and connects reset_driver and reset_sequencer. The monitor is always created, regardless of the mode.
- connect_phase: connects reset_sequencer to reset_driver, only when the agent is active.

●.1.4 Agent for the APB interface

The APB agent is the UVM component responsible for managing the APB interface inside the testbench. It encapsulates the three main components associated with a standard UVM interface:

- **apb_driver** – sends transactions to the APB interface (active mode only).
- **apb_monitor** – observes the signals on the bus and extracts transactions for verification.
- **apb_sequencer** – receives transactions generated by sequences (uvm_sequence) and forwards them to the driver.

Operating modes:

Active (UVM_ACTIVE) – the agent instantiates all three components and connects sequencer to driver.

Passive (UVM_PASSIVE) – only the monitor is instantiated, allowing passive interface verification without influencing the traffic.

UVM phases used:

build_phase: instantiates the components based on the active/passive mode.
connect_phase: connects sequencer and driver (active mode only).

●.1 LIST OF ITEMS TO VERIFY

Table 5. Checkers across the verification interfaces

Checker	Description
paddr	Verifies that the write targets the expected address
pwdata	Verifies that the data sequence is functionally correct
prdata	Verifies that the returned result matches the expectation
result	Verifies the correctness of the result

●.2 FUNCTIONAL COVERAGE ELEMENTS

The *coverage* can be listed as a list or table (each coverage group in its own table). They are also organized by components/interfaces.

Cover Point	Description
Cov_reg	All register addresses (0 - 15)
Cov_delay	Delay value between two transactions (0 - 15)
Cov_pwdata_addr	All register addresses as seen in Pwdata.
Cov_pwdata_op1 / Cov_pwdata_op2	Operand values as seen in Pwdata.
Cov_pwdata_const	Constant value as seen in Pwdata.
Cov_pwdata_shift	Shift value as seen in Pwdata.
Cov_opcode	Opcode values as seen in Pwdata.
Cov_data_read	Prdata value associated with the requested address.
Cov_pwrite	Pwrite value, write or read.
Cov_result	Result value on the Result bus.
Cov_state	State value, RUN or FREEZE.
Cov_reset_n	Reset_n value.

●.3 VERIFICATION SCENARIOS AND TESTS

Table 7. List of scenarios designed for the DUT verification

Scenario title	Description of the proposed scenario
Large operand values (Overflow)	We test how the unit behaves when extremely large values are used for operand 1 (op1) and operand 2 (op2), which can cause overflow during an addition operation.
Shift operation with a large value	We check the shift operation behavior when the operand has a large value and the shift factor is also large.
Reset asserted during an operation	We check the unit's behavior when the reset signal (reset_n) is asserted during a compute operation.
Read or write to an invalid register	We test what happens when a read or write is attempted on an invalid register, or on a register that is not available for read/write operations.
Use of a very large constant in an add operation	We check the behavior when a very large constant is used in an add operation with an operand.
Logic operation on invalid values	We test logic operations on invalid data (for example, incompatible operands).
AND logic operations	Testing the AND logic operation between two bit sets.
OR logic operations	Testing the OR logic operation between two bit sets.
Left shift	Testing the left shift operation on a bit set.
Left shift with large operand	Testing the left shift operation when the operand has a large value and the shift factor is maximum (3).
Equality compare	Verifying the compare function for equality between two values.
Greater-than compare	Testing the compare function to determine whether one number is greater than another.
Less-than compare	Testing the compare function to determine whether one number is less than another.
Operations with negative numbers	Testing the ALU behavior with negative operands.
Invalid address	Testing the DUT at an invalid address.

Invalid opcode	Testing the DUT with an invalid opcode.
B2B transactions	Testing the DUT under back-to-back transactions.
Transaction delays	Testing the DUT under transaction delays.

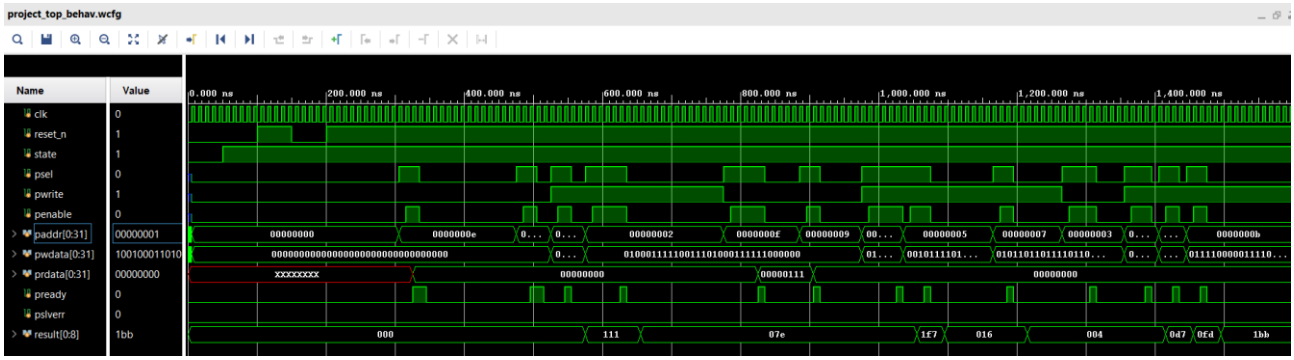
Table 8. Tests implemented for DUT verification

Name of the file containing the test	Description of the implemented test
first_test	This test exercises a simple write/read transaction sequence on a few registers
test_opcode_check_full	This test walks through every valid opcode (0001..1001) and checks the result.
test_write_all_read_all_error	This test writes and reads all register addresses plus one out-of-range address.
test_write_all_read_all	This test writes and reads all register addresses.
test_full_random	This test issues constrained-random APB transactions exercising every opcode.
test_opcode_error_check_with_0000	This test issues opcode 0000 (not implemented), which must raise an error on the output.
test_opcode_check_full_half_reset	This test executes every valid opcode operation with an extra reset in the middle, to check that the functionality still runs correctly afterwards.
test_full_random_with_reset	This test provides random values to every sequence, with a reset injected between them.

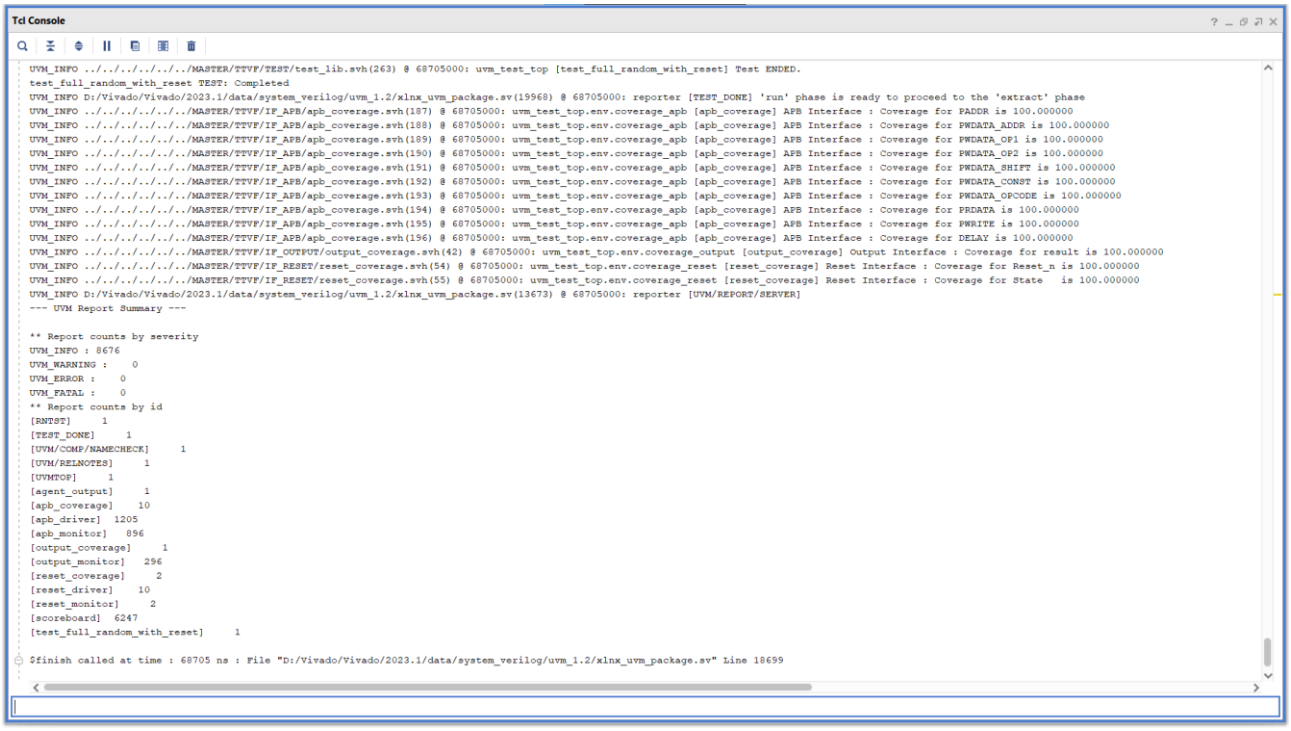
```
run_test("first_test_reg");
run_test("first_test_opcode");
run_test("test_write_all_read_all");
run_test("test_write_all_read_all_error");
run_test("test_opcode_random");
run_test("test_opcode_error_check_with_0000");
run_test("test_opcode_check_full");
run_test("test_opcode_check_full_half_reset");
run_test("test_full_random");
run_test("test_full_random_with_reset");
```

●.4 RESULTS OBTAINED

The results cover functional coverage, overall project progress and other points of interest.



After running test test_full_random_with_reset, a value of 100% was reached for the functional coverage element related to the latency values.



All results were obtained from running a single stress test. This is the final output of that test; each situation is described below:

```
APB Interface : Coverage for PADDR is 100.000000
APB Interface : Coverage for PWDATA_ADDR is 100.000000
APB Interface : Coverage for PWDATA_OP1 is 100.000000
APB Interface : Coverage for PWDATA_OP2 is 100.000000
APB Interface : Coverage for PWDATA_SHIFT is 100.000000
APB Interface : Coverage for PWDATA_CONST is 100.000000
APB Interface : Coverage for PWDATA_OPCODE is 100.000000
APB Interface : Coverage for PRDATA is 100.000000
APB Interface : Coverage for PWRITE is 100.000000
APB Interface : Coverage for DELAY is 100.000000
```

The APB interface was fully covered; no assertion failed and the functional coverage of the data reached its targets (Functional Coverage).

```
Output Interface : Coverage for result is 100.000000
```

The Output interface, which is passive, could only have its result checked; the result was split into 10 value bins, from the minimum up to the maximum on 9 bits.

```
Reset Interface : Coverage for Reset_n is 100.000000
Reset Interface : Coverage for State   is 100.000000
```

The Reset interface, which has only two signals, was checked to make sure both values (0 and 1) are reached.